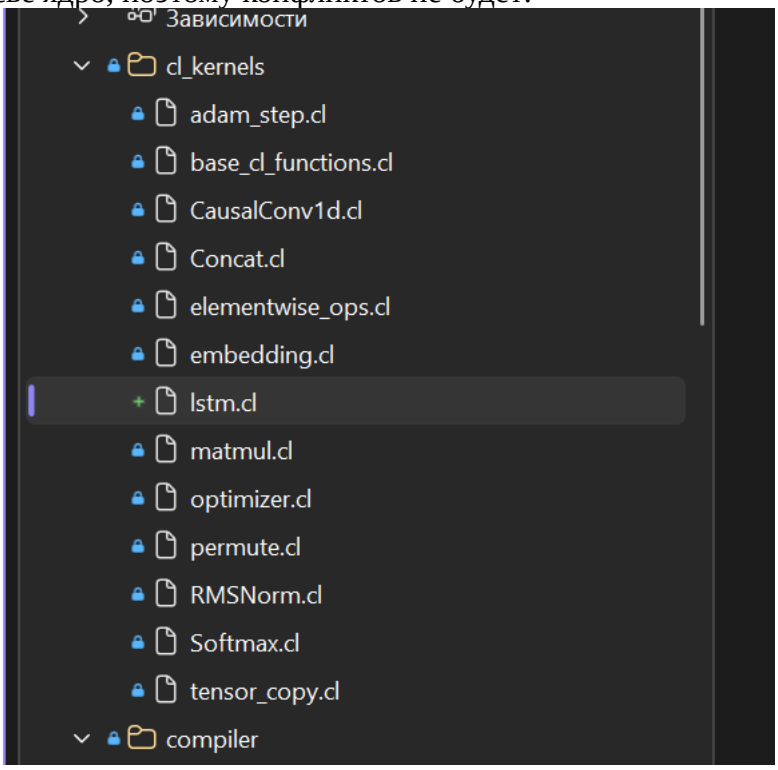
Скажите, какой подход к **аккумуляции матриц градиентов (W и U)** вам ближе — использовать атомарные функции прямо в ядре или выносить сложение батчей в отдельный шаг редукции?

Использовать атомарные функции прямо в ядре.
У аждого объекта све ядро, поэтому конфликтов не будет:



Теперь переходим к nnclglob.

```
using NOpenCL;
using System;
using System.Collections.Generic;
using System.Text;

namespace cstorchcl.core
{
    /// <summary>
    /// Основной класс тензора библиотеки cstorchcl.
    /// Объединяет в себе буфер данных на GPU (OpenCL) и контекст графа автодифференцирования.
    /// </summary>
    public class Tensor : IDisposable
    {
        //
=============================================================================
        // МЕТАДАННЫЕ И МАССИВЫ
        //
=============================================================================
```

```
        /// <summary>
        /// Форма тензора (размеры по осям, например: [Batch, Head, SeqLen, Dim]).
        /// </summary>
        public int[] Shape { get; init; }

        /// <summary>
        /// Общее количество элементов в тензоре (произведение всех элементов Shape).
        /// </summary>
        public int Size { get; init; }

        /// <summary>
        /// Флаг, указывающий, нужно ли вычислять градиент для этого тензора (участие в обучении).
        /// </summary>
        public bool RequiresGrad { get; set; }

        //
================================================================================
        // OPENCL РЕСУРСЫ (ДАННЫЕ НА GPU)
        //
================================================================================

        /// <summary>
        /// Низкоуровневый буфер OpenCL, хранящий значения тензора (Forward pass).
        /// </summary>
        public NOpenCL.Buffer DataBuffer { get; private set; }

        /// <summary>
        /// Буфер OpenCL, хранящий градиенты тензора (Backward pass).
        /// Инициализируется лениво только если RequiresGrad = true.
        /// </summary>
        public Tensor? Grad { get; set; }

        //
================================================================================
        // КОНТЕКСТ АВТОДИФФЕРЕНЦИРОВАНИЯ (AUTOGRAD)
        //
================================================================================

        /// <summary>
        /// Ссылка на операцию (Op), которая создала данный тензор в результате вычислений.
        /// Если null — тензор является листовым (leaf node), например, веса модели или входные данные.
        /// </summary>
        public object? Creator { get; set; } // В будущем заменим object на интерфейс IOp или базовый класс Op
```

```
        // Внутренний трекер очистки памяти
        private bool _isDisposed = false;

        //
==========================================================================
==
        // КОНСТРУКТОРЫ
        //
==========================================================================
==

        /// <summary>
        /// Создает экземпляр тензора с выделением памяти на GPU.
        /// </summary>
        /// <param name="shape">Форма тензора.</param>
        /// <param name="context">Текущий контекст OpenCL, в котором выделяется
память.</param>
        /// <param name="flags">Флаги аллокации памяти OpenCL (по умолчанию
ReadWrite).</param>
        /// <param name="requiresGrad">Флаг необходимости расчета градиентов.</param>
        public Tensor(int[] shape, Context context, MemoryFlags flags = MemoryFlags.ReadWrite,
bool requiresGrad = false)
        {
            if (shape == null || shape.Length == 0)
                throw new ArgumentException("Форма тензора должна содержать хотя бы одно
измерение.", nameof(shape));

            if (context == null)
                throw new ArgumentNullException(nameof(context), "Контекст OpenCL не может
быть null.");

            Shape = (int[])shape.Clone();
            Size = CalculateSize(shape);
            RequiresGrad = requiresGrad;
            Strides = CalculateStrides(shape);
            // Вычисляем размер буфера в байтах. Сейчас жестко завязано на float (4 байта)
            long sizeInBytes = (long)Size * sizeof(float);

            try
            {
                // Аллокация памяти на девайсе через NOpenCL
                DataBuffer = context.CreateBuffer(flags, sizeInBytes);
            }
            catch (Exception ex)
            {
                // Логируем контекст ошибки, упрощая отладку OOM (Out Of Memory) на GPU
                throw new InvalidOperationException(
                    $"Ошибка выделения GPU буфера размером {sizeInBytes} байт для формы
[{string.Join(", ", Shape)}].", ex);
            }
        }
        /// <summary>
```

```
        /// Создает экземпляр тензора и сразу загружает в него данные из C# массива float[] во VRAM.
        /// </summary>
        /// <param name="data">Исходный массив данных на CPU (хосте).</param>
        /// <param name="shape">Форма тензора.</param>
        /// <param name="context">Текущий контекст OpenCL.</param>
        /// <param name="flags">Флаги памяти OpenCL (по умолчанию ReadWrite).</param>
        /// <param name="requiresGrad">Флаг необходимости расчета градиентов.</param>
        public unsafe Tensor(float[] data, int[] shape, Context context, MemoryFlags flags = MemoryFlags.ReadWrite, bool requiresGrad = false)
        {
            if (data == null)
                throw new ArgumentNullException(nameof(data), "Массив данных не может быть null.");

            if (shape == null || shape.Length == 0)
                throw new ArgumentException("Форма тензора должна содержать хотя бы одно измерение.", nameof(shape));

            if (context == null)
                throw new ArgumentNullException(nameof(context), "Контекст OpenCL не может быть null.");

            Shape = (int[])shape.Clone();
            Size = CalculateSize(shape);
            RequiresGrad = requiresGrad;
            Strides = CalculateStrides(shape);
            // Важнейшая валидация: проверяем, совпадает ли физический размер массива с логической формой
            if (data.Length != Size)
            {
                throw new ArgumentException(
                    $"Несоответствие размера данных форме тензора. Ожидалось элементов: {Size}, получено: {data.Length}.", nameof(data));
            }

            long sizeInBytes = (long)Size * sizeof(float);

            try
            {
                // Для оптимизации пересылки мы можем задействовать флаг CopyHostPtr,
                // если API OpenCL поддерживает прямую аллокацию с копированием.
                // Но для гарантированной стабильности и гибкости флагов (например, если передали ReadOnly),
                // мы выделяем чистый буфер и сразу пишем в него через текущую глобальную очередь.
                DataBuffer = context.CreateBuffer(flags, sizeInBytes);

                // Получаем текущую очередь команд из регистра для немедленной записи данных
                CommandQueue queue = DeviceQueueRegistry.GetCurrent();
```

```
            // Используем fixed-указатель для безопасного получения нативного адреса управляемого массива
            fixed (float* pData = data)
            {
                // Синхронно пишем данные во VRAM (blocking_write = true)
                queue.EnqueueWriteBuffer(DataBuffer, true, 0, sizeInBytes, (IntPtr)pData);
            }

            // Финализируем операцию записи на устройстве
            queue.Finish();
        }
        catch (Exception ex)
        {
            throw new InvalidOperationException(
                $"Ошибка инициализации GPU буфера данными для формы [{string.Join(", ", Shape)}].", ex);
        }
    }

    // ================================================================================
    // ВСПОМОГАТЕЛЬНЫЕ МЕТОДЫ
    // ================================================================================

    /// <summary>
    /// Вычисляет общее количество элементов на основе формы тензора.
    /// </summary>
    private static int CalculateSize(int[] shape)
    {
        int total = 1;
        foreach (var dim in shape)
        {
            if (dim <= 0)
                throw new ArgumentException($"Размерность оси не может быть меньше или равна нулю: {dim}");
            total *= dim;
        }
        return total;
    }

    /// <summary>
    /// Инициализирует буфер градиента (если он еще не создан и RequiresGrad = true).
    /// </summary>
    /// <param name="context">Контекст OpenCL.</param>
    public void InitGradient(Context context)
    {
        // Исправление 1: Выходим, если расчет градиентов НЕ требуется
        if (!RequiresGrad) return;
```

```
            // Исправление 2: Правильная проверка nullable-объекта Tensor на null
            if (Grad == null)
            {
                // Градиент всегда имеет ту же форму, что и сам тензор
                Grad = new Tensor(Shape, context, MemoryFlags.ReadWrite, requiresGrad: false);

                // ВАЖНО: В Autograd градиенты перед началом обратного прохода обнуляются.
                // Заполнение нулями (Zero-init) должно происходить через OpenCL Queue (команда fill или запись массива).
            }
        }


        //
============================================================================
==
        // УПРАВЛЕНИЕ ПАМЯТЬЮ (ИСПРАВЛЕНИЕ УТЕЧЕК)
        //
============================================================================
==

        /// <summary>
        /// Освобождает неуправляемые ресурсы OpenCL буфера.
        /// Крайне важно для предотвращения утечек VRAM на GPU.
        /// </summary>
        public void Dispose()
        {
            Dispose(true);
            GC.SuppressFinalize(this);
        }

        protected virtual void Dispose(bool disposing)
        {
            if (_isDisposed) return;

            if (disposing)
            {
                // Освобождаем управляемый объект градиента, если он есть
                if (Grad != null)
                {
                    Grad.Dispose();
                    Grad = null;
                }
            }

            // Освобождаем низкоуровневый буфер OpenCL данных
            if (DataBuffer != null)
            {
                try
                {
                    DataBuffer.Dispose();
                }
```

```
            catch (Exception ex)
            {
                // Запись в диагностику, если буфер уже был освобожден контекстом
                System.Diagnostics.Debug.WriteLine($"Предупреждение при Dispose DataBuffer: {ex.Message}");
            }
        }

        _isDisposed = true;
    }

    /// <summary>
    /// Деструктор на случай, если пользователь забыл вызвать Dispose вручную.
    /// </summary>
    ~Tensor()
    {
        Dispose(false);
    }
    /// <summary>
    /// Шаги (strides) тензора для каждой оси. Показывают смещение в плоском массиве.
    /// Например, для Shape [2, 3, 4] Strides будут.
    /// </summary>
    public int[] Strides { get; init; }

    // Добавьте вызов этого метода в конструкторы класса Tensor сразу после Size = CalculateSize(shape);
    // Strides = CalculateStrides(Shape);

    /// <summary>
    /// Вычисляет классические непрерывные (contiguous) шаги осей на основе формы тензора.
    /// </summary>
    private static int[] CalculateStrides(int[] shape)
    {
        int[] strides = new int[shape.Length];
        int currentStride = 1;

        // Идем с конца формы к началу
        for (int i = shape.Length - 1; i >= 0; i--)
        {
            strides[i] = currentStride;
            currentStride *= shape[i];
        }
        return strides;
    }

  }
}
```



Тперь переходим к реализации методов Lstm и LstmBackward в NnCLGlob.
Учитовай следующие особенности архитектуры (На примере softmax):

```
/// <summary>
/// Глобальная безопасная операция Softmax (Forward).
/// Рассчитана на работу со скорами внимания размерности [B * H * L, SEQ_LEN].
/// </summary>
public Tensor Softmax(Tensor x, int dim = -1)
{
    // Эквивалент проверки размерности в Python: assert dim == -1 or dim == len(x.shape) - 1
    int targetDim = dim == -1 ? x.Shape.Length - 1 : dim;
    if (targetDim != x.Shape.Length - 1)
    {
        throw new NotSupportedException("Текущая реализация ядра Softmax поддерживает редукцию только по последней оси.");
    }

    int seqLen = x.Shape[x.Shape.Length - 1]; // Последняя ось
    int numRows = x.Size / seqLen;            // Вычисляем количество строк

    // Создаем выходной тензор той же формы во VRAM (RequiresGrad пока передаем как у исходного)
    Tensor y = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: x.RequiresGrad);

    // Настройка сетки потоков (Сетка идентична вашей конфигурации на Python)
    IntPtr localSize = new IntPtr(256);
    IntPtr globalSize = new IntPtr(numRows * 256);

    // Получаем глобальную активную очередь выполнения задач из нашего реестра
    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();

    // Передаем аргументы в Си-ядро по порядку индексов (0, 1, 2, 3):
    // kernel void softmax_forward(global const float* X, global float* Y, const int seq_len)
    _kernelSoftmaxForward.Arguments[0].SetValue(x.DataBuffer);
    _kernelSoftmaxForward.Arguments[1].SetValue(y.DataBuffer);
    _kernelSoftmaxForward.Arguments[2].SetValue(seqLen);

    // Запуск выполнения ядра на GPU
```

```
    activeQueue.EnqueueNDRangeKernel(_kernelSoftmaxForward, null, new[] {
globalSize }, new[] { localSize });

    // Синхронизация: гарантирует завершение вычислений перед чтением
    activeQueue.Finish();

    // Если включен режим Autograd и требуется расчет градиента, привязываем
контекст создателя к тензору
    if (y.RequiresGrad)
    {
        // Создаем операцию, передавая входы и выходы
        y.Creator = new SoftmaxOp(x, y, this);
    }

    return y;
}
/// <summary>
/// Обратный проход глобальной операции Softmax.
/// Использует уже вычисленный forward-выход тензора Y для экономии памяти VRAM.
/// </summary>
/// <param name="gradOutput">Пришедший сверху градиент dL/dY [B * H * L,
SEQ_LEN]</param>
/// <param name="y">Исходный выход прямого прохода Softmax (вероятности) [B * H
* L, SEQ_LEN]</param>
/// <returns>Выходной градиент по входу dL/dX [B * H * L, SEQ_LEN]</returns>
public Tensor SoftmaxBackward(Tensor gradOutput, Tensor y)
{
    int seqLen = y.Shape[y.Shape.Length - 1];
    int numRows = y.Size / seqLen;

    // Выделяем тензор для результирующего градиента по входу (по форме
совпадает с Y)
    Tensor gradInput = new Tensor(y.Shape, _computeCtx.Context,
MemoryFlags.ReadWrite, requiresGrad: false);

    // Настройка сетки потоков (идентично forward шагу)
    IntPtr localSize = new IntPtr(256);
    IntPtr globalSize = new IntPtr(numRows * 256);

    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
    int t = 0;
    // Передаем аргументы в Си-ядро softmax_backward:
    // kernel void softmax_backward(global const float* grad_output, global
const float* Y, global float* grad_input, const int seq_len)
    _kernelSoftmaxBackward.Arguments[t++].SetValue(gradOutput.DataBuffer);
    _kernelSoftmaxBackward.Arguments[t++].SetValue(y.DataBuffer);
    _kernelSoftmaxBackward.Arguments[t++].SetValue(gradInput.DataBuffer);
    _kernelSoftmaxBackward.Arguments[t++].SetValue(seqLen);

    // Запуск ядра
    activeQueue.EnqueueNDRangeKernel(_kernelSoftmaxBackward, null, new[] {
globalSize }, new[] { localSize });
    activeQueue.Finish();

    return gradInput;
}
```

```
using System;
using NOpenCL;
using cstorchcl.compiler;

namespace cstorchcl.core
{
    /// <summary>
    /// Операция графа автодифференцирования для функции активации Softmax.
    /// </summary>
    public class SoftmaxOp : Op
    {
        private readonly NnClGlob _nnClGlob;

        /// <summary>
        /// Создает узел операции Softmax в графе вычислений.
        /// </summary>
        /// <param name="input">Входной тензор X.</param>
        /// <param name="output">Выходной тензор Y (результат прямого
прохода).</param>
        /// <param name="nnClGlob">Диспетчер глобальных ядер.</param>
        public SoftmaxOp(Tensor input, Tensor output, NnClGlob nnClGlob)
        {
            Inputs = new[] { input };
            Output = output;
            _nnClGlob = nnClGlob ?? throw new
ArgumentNullException(nameof(nnClGlob));
        }

        /// <summary>
        /// Вычисляет градиент функции Softmax по входу X.
        /// </summary>
        public override Tensor[] Backward(Tensor gradOutput, Context context)
        {
            if (gradOutput == null)
                throw new ArgumentNullException(nameof(gradOutput), "Градиент,
пришедший сверху, не может быть null.");

            // Для расчета градиента Softmax нам нужен сохраненный выход прямого
прохода (Output)
            // Формула требует поэлементных операций над пришедшим градиентом и
вероятностями Softmax
            Tensor gradInput = _nnClGlob.SoftmaxBackward(gradOutput, Output);

            // Возвращаем массив градиентов по количеству входов. У Softmax один
вход X, поэтому массив из 1 элемента.
            return new[] { gradInput };
        }
    }
}
```